

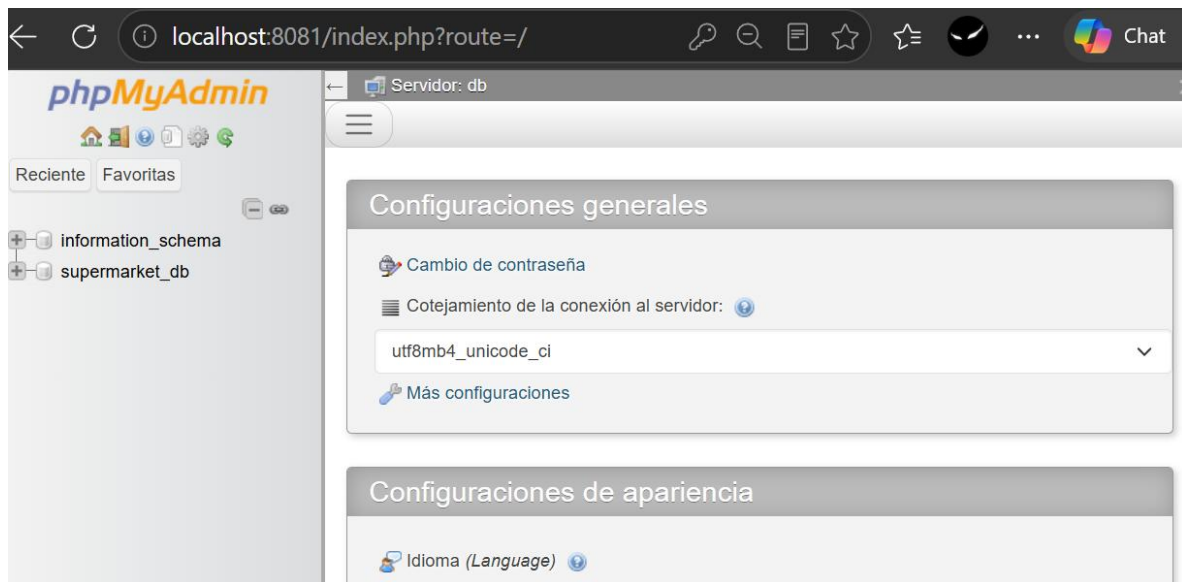
Taller #1 Docker

Componente #1

```
PS C:\Users\perez\Local\Projects\GitHub\supermarket_db> docker build -t supermarket_db .
[+] Building 21.2s (5/5) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 183B                             0.0s
=> [internal] load metadata for docker.io/library/mysql:5.7     4.6s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                   0.0s
=> [1/1] FROM docker.io/library/mysql:5.7@sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb 7.2s
=> => resolve docker.io/library/mysql:5.7@sha256:4bc6bc963e6d8443453676cae56536f4b8156d78bae03c0145cbe47c2aad73bb 0.0s

PS C:\Users\perez\Local\Projects\GitHub\supermarket_db> docker run -d -t -i -p 3306:3306 --name supermarket_db supermarket_db
4232ebfa666a41b86f1aed3e9956fc1e24e5e054b98e69fb074de27ebd261f3d

PS C:\Users\perez\Local\Projects\GitHub\supermarket_db> docker run --name phpmyadmin -d --link supermarket_db:db -p 8081:80
phpmyadmin
Unable to find image 'phpmyadmin:latest' locally
latest: Pulling from library/phpmyadmin
9cd7b4cda6ad: Pulling fs layer
```



Este componente corresponde a una base de datos MySQL ejecutándose dentro de un contenedor Docker. Su función es almacenar de forma persistente toda la información del sistema, como las categorías y los productos. Durante el despliegue se configuró el usuario, la contraseña y la base de datos mediante variables de entorno definidas en el Dockerfile.

Componente #2

```
PS C:\Users\perez\Local\ProjectsGitHub\supermarket_ms> docker ps
```

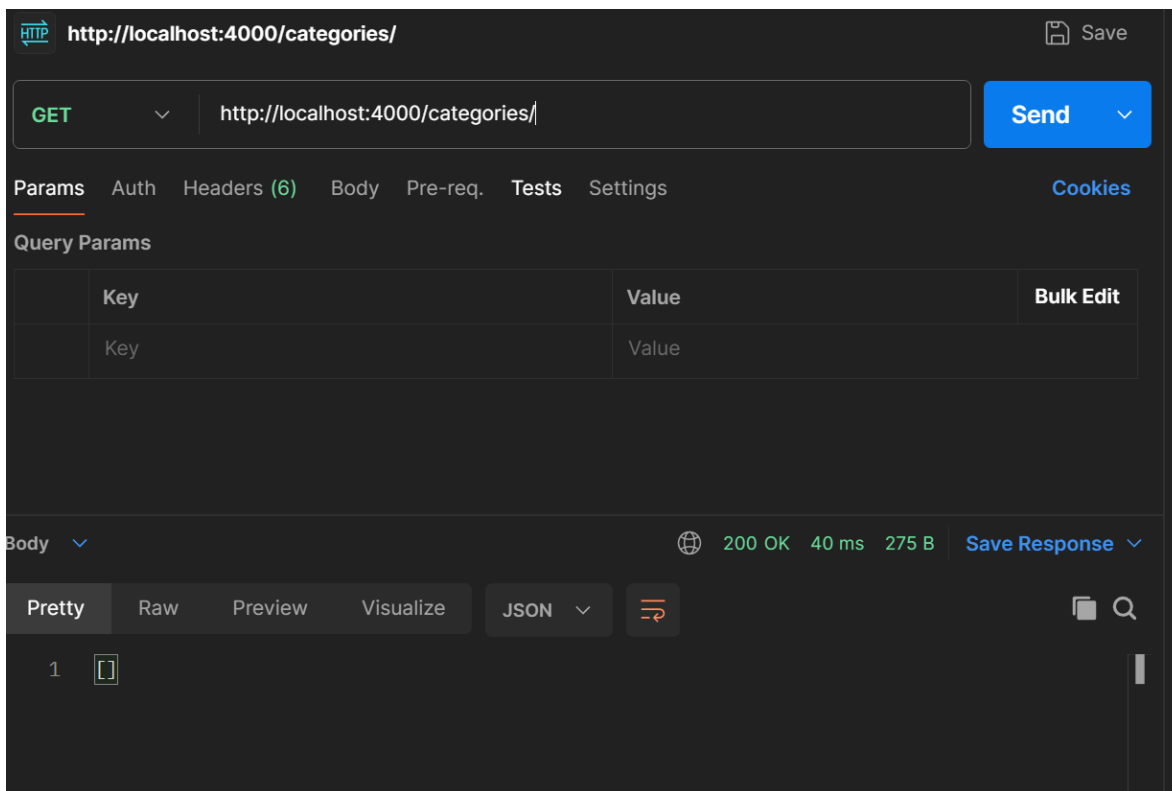
CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d0778dbad02a	supermarket_ms	"sh -c 'python manag..."	38 seconds ago	Up 37 seconds	0.0.0.0:4000->4000/tcp, [::]:4000->4000/tcp
ee7305f3b225	unruffled_lichterman	"/docker-entrypoint.s..."	16 minutes ago	Up 16 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
4232ebfa666a	phpmyadmin	"docker-entrypoint.s..."	17 minutes ago	Up 17 minutes	0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp

```
PS C:\Users\perez\Local\ProjectsGitHub\supermarket_ms>
```

```
August 03, 2026 - 12:49:25
Django version 3.1, using settings 'project.settings'
Starting development server at http://0.0.0.0:4000/
Quit the server with CONTROL-C.
Not Found: /
[03/Aug/2026 12:52:31] "GET / HTTP/1.1" 404 3254
Not Found: /favicon.ico
[03/Aug/2026 12:52:31] "GET /favicon.ico HTTP/1.1" 404 3305
```

Este componente es un microservicio desarrollado en Django y Django REST Framework. Su función es exponer una API REST que permite realizar operaciones CRUD sobre las entidades del sistema, comunicándose con la base de datos MySQL mediante el conector mysqlclient. La conexión se realiza utilizando variables de entorno como DB_HOST, DB_NAME, DB_USER y DB_PASSWORD.

Componente 3



El método **GET** consulta las categorías almacenadas en la base de datos.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:4000/categories/`
- Method:** **POST**
- Request Body (JSON):**

```
1 {
2   "name": "Electrónicos",
3   "description": "Categoría de productos electrónicos y
4                 dispositivos tecnológicos"
5 }
```
- Response Status:** **201 Created** (77 ms, 394 B)
- Response Body (JSON):**

```
1 {
2   "id": 4,
3   "name": "Electrónicos",
4   "description": "Categoría de productos electrónicos y
5                 dispositivos tecnológicos"
6 }
```

El método **POST** crea una nueva categoría. Se envía un objeto JSON con el nombre y la descripción, y el microservicio almacena la información en la base de datos.

 **http://localhost:4000/categories/4**  Save  Share 

GET

http://localhost:4000/categories/4

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK • 45 ms • 396 B •  ...

{ } JSON

Preview

Visualize

1 {

2 " id": 4,

3 " name": "Electrónicos",

4 " description": "Categoría de productos electrónicos y

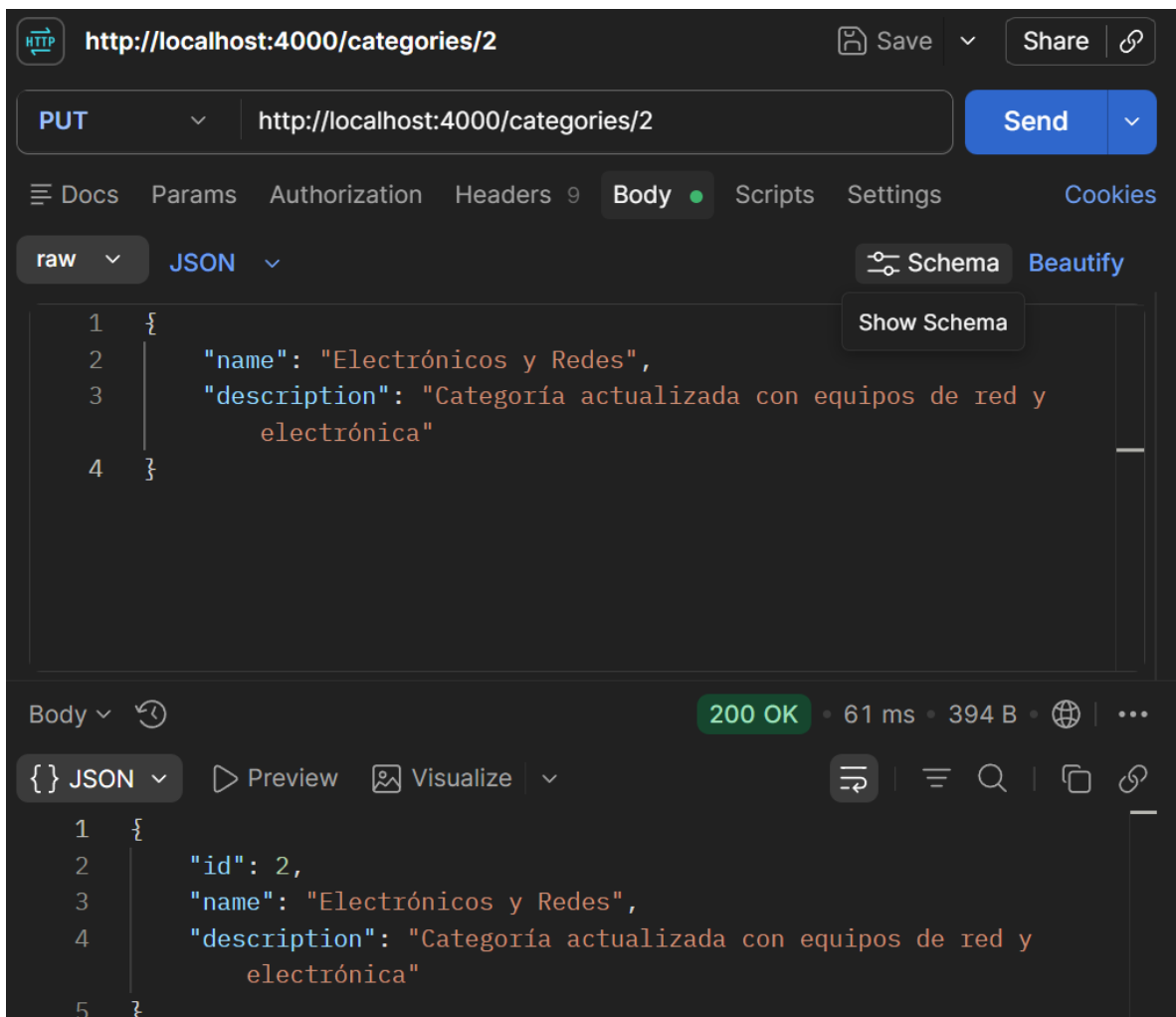
5 dispositivos tecnológicos"

5 }

The screenshot shows a web browser interface with the address bar displaying `http://localhost:4000/categories/`. The browser's developer tools are open, showing the 'Params' tab for the query parameters. Below this, the 'Body' tab is selected, displaying the response body in JSON format. The response status is `200 OK` with a response time of `44 ms` and a size of `549 B`. The JSON data represents two categories:

```
{
  "id": 3,
  "name": "Cat. Jeisson",
  "description": "Categoría creada para el curso"
},
{
  "id": 4,
  "name": "Electrónicos",
  "description": "Categoría de productos electrónicos y dispositivos tecnológicos"
}
```

El método **GET** consulta las categorías almacenadas en la base de datos.



El método **PUT** actualiza la información de una categoría existente.

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:4000/products/`
- Method:** `POST`
- Body (Request):**

```
1  {
2    "name": "Switch",
3    "description": "dispositivo de red que permite conectar
                     diferentes subredes y realizar enrutamiento IP a alta
                     velocidad",
4    "unit_measurement": "Units",
5    "quantity": 5,
6    "category": 2
7  }
```
- Status:** `201 Created` (62 ms, 588 B)
- Body (Response):**

```
1  {
2    "id": 2,
3    "name": "Switch",
4    "description": "dispositivo de red que permite conectar
                     diferentes subredes y realizar enrutamiento IP a alta
                     velocidad",
5    "category": {
6      "id": 2,
7      "name": "Electrónicos y Redes",
8      "description": "Categoría actualizada con equipos de red y
                     electrónica"
```

Creamos un nuevo producto

The screenshot shows a web browser's developer tools interface. At the top, the address bar displays `http://localhost:4000/products/1`. Below it, the 'GET' method is selected, and the URL `http://localhost:4000/products/1` is entered. The 'Send' button is visible. The 'Params' tab is active, showing a table with columns 'Key', 'Value', and 'Description'. The 'Body' tab is also visible, showing a '200 OK' status with a response time of 51 ms and a size of 593 B. The response body is displayed in JSON format:

```
1 {
2   "id": 1,
3   "name": "Switch L3",
4   "description": "dispositivo de red que permite conectar
5                  diferentes subredes y realizar enrutamiento IP a alta
6                  velocidad",
7   "category": {
8     "id": 2,
9     "name": "Electrónicos y Redes",
10    "description": "Categoría actualizada con equipos de red y
```

Obtenemos los productos de la DB

The screenshot shows a web browser's developer tools interface. At the top, the address bar displays `http://localhost:4000/products/`. Below it, the **GET** method is selected, and the **Send** button is visible. The **Params** tab is active, showing a table for Query Params.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Below the table, the **Body** tab is selected, showing a **200 OK** status with a response time of 54 ms and a size of 897 B. The response is in **JSON** format, and the **Preview** button is visible. The JSON body is displayed as follows:

```
1  [
2    {
3      "id": 1,
4      "name": "Switch L3",
5      "description": "dispositivo de red que permite conectar
6                      diferentes subredes y realizar enrutamiento IP a alta
7                      velocidad",
8      "category": {
9        "id": 2,
10       "name": "Electrónicos y Redes",
```

Obtenemos los productos de la DB

The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:4000/products/1`
- Method:** `PUT`
- Body (Request):**

```
1  {
2    "name": "Switch Cisco",
3    "description": "Switch de red empresarial con capacidad de
4                    enrutamiento avanzado",
5    "unit_measurement": "Units",
6    "quantity": 10,
7    "category": 2
8  }
```
- Status:** `200 OK` (71 ms, 558 B)
- Body (Response):**

```
1  {
2    "id": 1,
3    "name": "Switch Cisco",
4    "description": "Switch de red empresarial con capacidad de
5                    enrutamiento avanzado",
6    "category": {
7      "id": 2,
8      "name": "Electrónicos y Redes",
9      "description": "Categoría actualizada con equipos de red y
10                     electrónica"
11    }
12  }
```

Actualizamos el producto 1

The screenshot shows a REST client interface with the following components:

- URL Bar:** Displays `http://localhost:4000/products/1` with a **DELETE** method selected and a **Send** button.
- Navigation Tabs:** Includes **Docs**, **Params**, **Authorization**, **Headers** (7), **Body**, **Scripts**, **Settings**, and **Cookies**.
- Query Params Table:**

Key	Value	Description	Bulk Edit	...
Key	Value	Description		
- Response Section:**
 - Status:** **204 No Content** (highlighted in green).
 - Details:** 59 ms • 256 B • [Globe icon] • [More icon]
 - Body View:** **Raw** (selected), **Preview**, **Visualize**.
 - Content:** The number `1` is visible in the raw view.

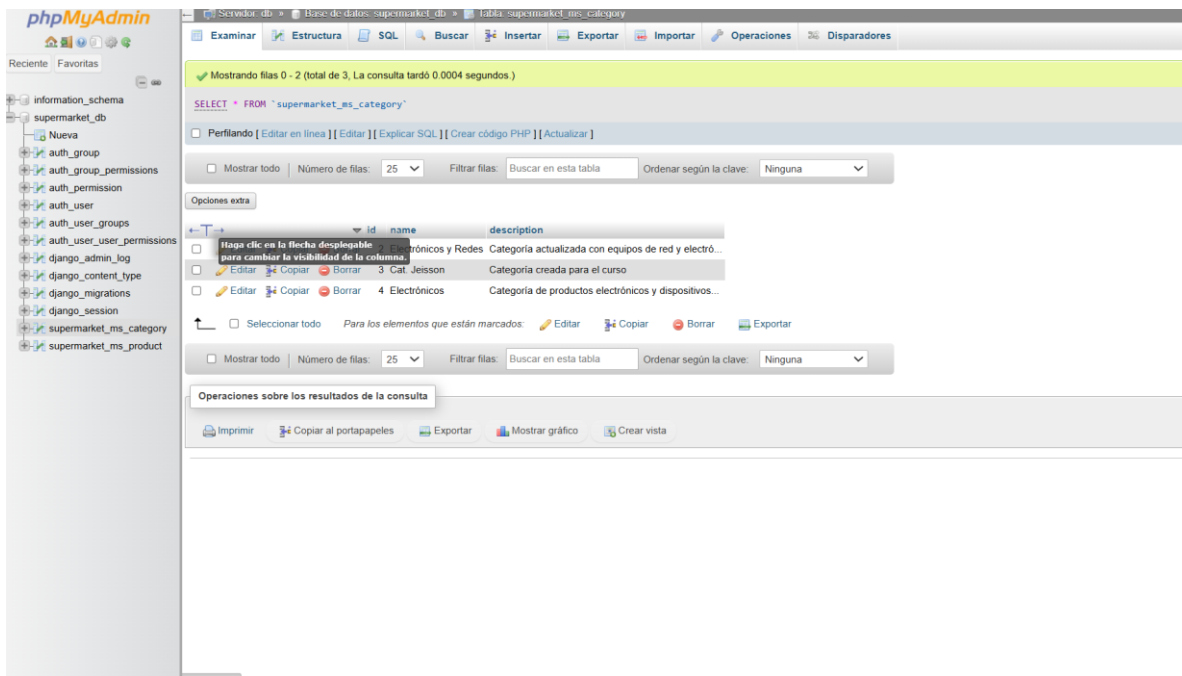
Se elimina el producto 1

```
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_supermarket_db |
+-----+
| auth_group                |
| auth_group_permissions    |
| auth_permission           |
| auth_user                 |
| auth_user_groups          |
| auth_user_user_permissions|
| django_admin_log          |
| django_content_type       |
| django_migrations         |
| django_session            |
| supermarket_ms_category   |
| supermarket_ms_product    |
+-----+
12 rows in set (0.00 sec)

mysql> SELECT * FROM supermarket_ms_category;
+----+-----+-----+
| id | name      | description |
+----+-----+-----+
| 2  | Cat. Jeisson | Categoría creada para el curso |
+----+-----+-----+
1 row in set (0.00 sec)

mysql> █
```

Después de ejecutar cada petición HTTP se verificó el estado de la tabla utilizando PhpMyAdmin y el cliente MySQL. Esto confirma que el microservicio interactúa correctamente con la base de datos y que las operaciones CRUD se ejecutan de forma satisfactoria.



phpMyAdmin

Reciente Favoritas

Information_schema
supermarket_db
 Nueva
 auth_group
 auth_group_permissions
 auth_permission
 auth_user
 auth_user_groups
 auth_user_user_permissions
 django_admin_log
 django_content_type
 django_migrations
 django_session
 supermarket_ms_category
 supermarket_ms_product

Mostrando filas 0 - 2 (total de 3, La consulta tardó 0.0004 segundos.)

SELECT * FROM `supermarket\_ms\_category`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

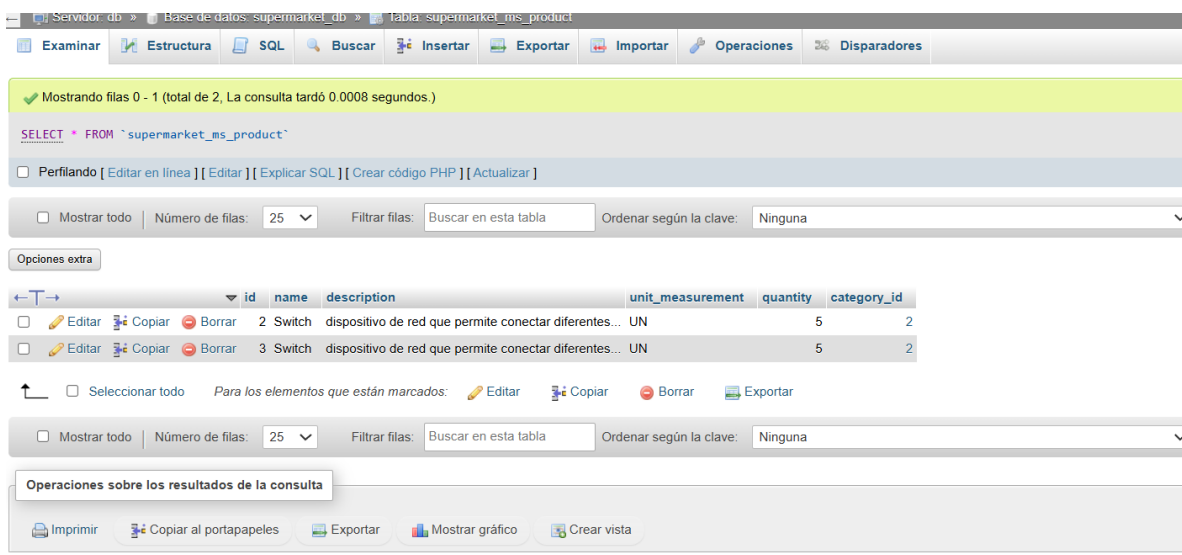
	id	name	description
☐	Haga clic en la flecha desplegable para cambiar la visibilidad de la columna	3	Electrónicos y Redes
☐	Editar Copiar Borrar	3	Cat Jeisson
☐	Editar Copiar Borrar	4	Electrónicos

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista



Mostrando filas 0 - 1 (total de 2, La consulta tardó 0.0008 segundos.)

SELECT * FROM `supermarket\_ms\_product`

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	id	name	description	unit_measurement	quantity	category_id
☐	Editar Copiar Borrar	2	Switch	dispositivo de red que permite conectar diferentes...	UN	5
☐	Editar Copiar Borrar	3	Switch	dispositivo de red que permite conectar diferentes...	UN	5

Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar

Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

Docker permitió empaquetar tanto la base de datos como el microservicio en contenedores independientes. Cada contenedor ejecuta un servicio específico y se comunica mediante la red de Docker.